CENG-567
REINFORCEMENT LEARNING

Lecture-4

Contextual Bandits

Dr. Osman GÖKALP

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap

What are Contextual Bandits? (CBs)

Definition: Advanced MAB where decisions (**actions**) are tailored to the observed **context** (x), like user age or location.

Role: Central to commercial **personalization and recommendation** (e.g., ad targeting).

Key Distinction from RL: Action leads to Reward, but **does NOT influence the next context**. No state transitions or long-term planning.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap

Why Function Approximation is CRITICAL?

We need to learn the action value Q(a | x).

Problem with Simple Tables (MAB approach)	Solution: Function Approximation
Exponential Context Growth: Too many feature combinations lead to **data fragmentation**.	Transforms CB into a **Supervised Learning** problem: Context x maps to Q(a)
Intractable: Not enough samples for unique context combinations.	Models (e.g., **Logistic Regression**) learn a **relationship** between x and Q.
No Knowledge Sharing: Similar contexts (e.g., age 56 vs. 57) treated as unrelated.	Enables **Knowledge Sharing** between similar contexts.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap

Learning $Q(a \mid x)$ with Regularized Logistic Regression

Model Choice: **Logistic Regression** is used to predict the click probability (a binary outcome) from the context x.

$$\text{Prediction} = \text{Sigmoid}(\text{Weights} * \text{Context} + \text{Bias})\})$$

Thompson Sampling (TS) Strategy: We use Regularized LR to maintain a **distribution over the model weights**, enabling probabilistic exploration.

1. The posterior is approximated as a Gaussian Distribution (**Mean** 'm', **Covariance** 'inverse of q').
2. For action selection, weights are **sampled** from this distribution.
3. **Exploration**: High model **uncertainty** (large covariance) results in more random samples, driving exploration.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap

Measuring Success: Regret Minimization

Regret: The loss from not choosing the optimal ad (a^*) for context x.

$$\text{Regret} = p_{a^*}(x_k) - p_a(x_k)$$

Goal: Minimize Total (Cumulative) Regret. A successful algorithm shows decelerating cumulative regret (the curve flattens).

Key Result: Thompson Sampling significantly outperformed epsilon-greedy and UCB in the case study, proving its efficiency for exploration.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Using Function Approximation for Action

The Challenge: Dynamic Ad Inventory 🔄

Previous examples assumed a **fixed set of ads** (actions/arms).

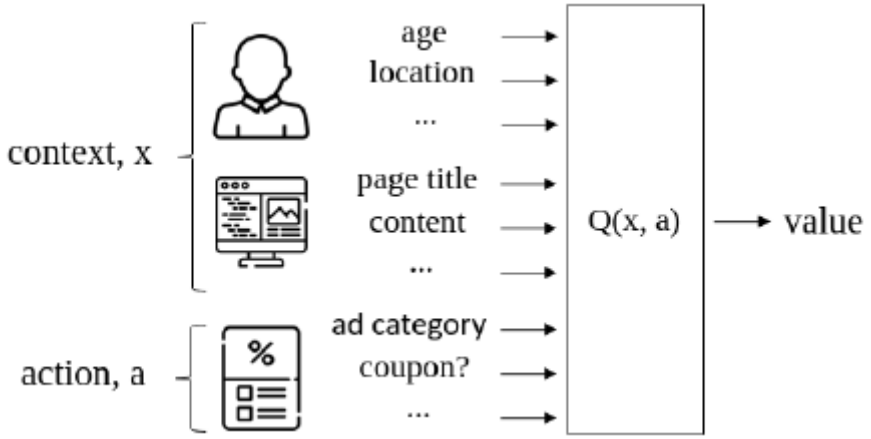
Modern systems (e.g., ad networks) have **dynamic Ad Inventory**—the available ads constantly change.

We cannot use a separate model for every possible ad, as the inventory is too large and volatile.

Solution: Single Model with Action Features

- To handle dynamic actions, we switch from a model per action to a **single, unified model**. This structure allows the single model to generalize and evaluate any ad ($\mathbf{a}$) based on the current context ($\mathbf{x}$).

Old Structure (Fixed Actions)	**New Structure (Dynamic Actions)**
Model: $Q_a(\mathbf{x})$	**Model:** $Q(\mathbf{x}, \mathbf{a})$
Inputs: Context ($\mathbf{x}$) only.	**Inputs:** Context ($\mathbf{x}$) **AND Ad Features ($\mathbf{a}$)**.
Decision: Lookup Q for all fixed ads.	**Decision: Input available Ad Features ($\mathbf{a}$) into the single model** to generate Q.

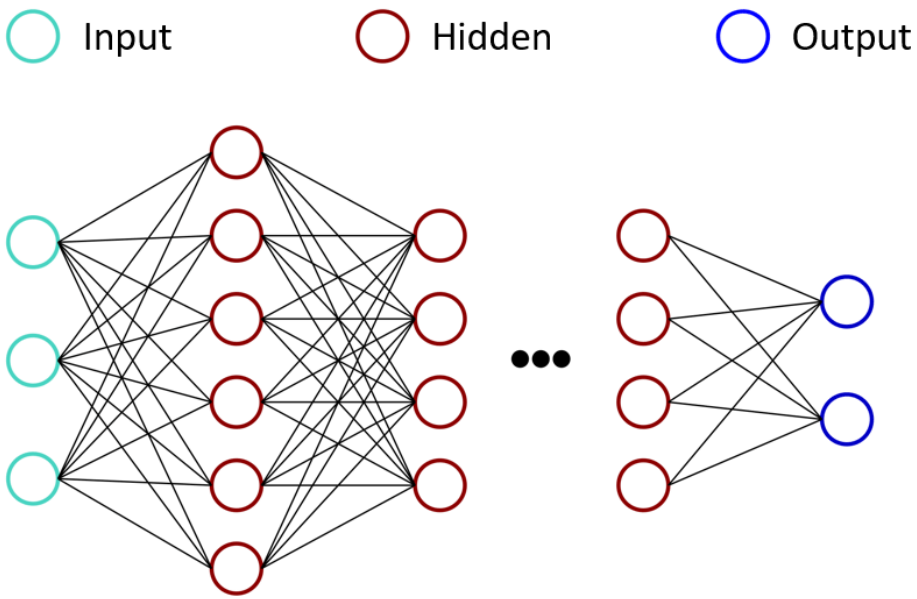


Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Complexity Demands Sophistication

The relationship between **User Context ($\mathbf{x}$), Website Content ($\mathbf{x}$), and Ad Features ($\mathbf{a}$)** is highly complex.

To accurately estimate the click probability (Q) from this combined input, and assuming we have enough data, the obvious choice for the function approximator is **Deep Neural Networks (DNNs)**.



Bu Fotoğraf, Bilinmeyen Yazar, CC BY altında lisanslanmıştır

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

The Thompson Sampling Intractability Problem with DNNs

Thompson Sampling (TS) is a competitive exploration strategy.

TS requires us to **sample from the posterior distribution** of the **model's parameters** (weights).

This sampling process is intractable (computationally impossible to calculate exactly) for complex models like **Deep Neural Networks (DNNs)**.

Solution: Approximate Bayesian Inference 💡

We must rely on approximation methods to estimate parameter uncertainty for TS.

Method: Monte Carlo Dropout (based on Gal et al., 2015).

Dropout Reminder: Dropout randomly deactivates neurons during training to prevent overfitting and increase generalization.

The Trick: Keep dropout active during inference (prediction) time.

Standard DNN Usage	Approximate Bayesian DNN Usage (for TS)
Dropout is **OFF** during prediction.	Dropout is **KEPT ON** during prediction.
Output is deterministic (single Q value).	Output **varies probabilistically** (multiple Q values).

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Case study – contextual online advertising with user data from the US Census

New Case Study: Contextual Advertising with Census Data 📊

Source Dataset: Modified **1994 US Census Income Dataset** (UCI Repository).

Goal: Adapt this real-world dataset to simulate a **Contextual Bandit (CB)** problem in an online advertising setting.

Dataset Information (Context Features) 👤

We will use detailed demographic and professional data on individuals to serve as the Context ($\mathbf{x}$) for ad decisions.

Category	Features Used
Demographic	Age, Race, Gender, Native Country
Socio-Economic	Work Class, Education, Marital Status, Relationship
Employment	Occupation, Work Hours per Week

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Case Study Scenario: Targeting Hidden Education Level 🎯

The Contextual Bandit Challenge

Ad Server Knowledge: The server knows most user attributes (Age, Occupation, Gender, etc.)—these form the known Context ($\mathbf{x}$).

Missing Information: The server does **NOT know** the user's **Education Level** (not part of the context).

Action (Ad) Knowledge: The **Ad Inventory** contains ads explicitly **targeting a specific education level** (e.g., 'College Ad', 'Elementary School Ad').

The Click Dynamic (Reward Structure)

The probability of a click is determined by the **match** between the ad's target audience and the user's actual, but hidden, education level.

High Probability: If the ad's **Target Education Level** perfectly matches the user's actual **Education Level**.

Decreasing Probability: The probability of a click **decreases gradually** as the discrepancy between the ad's target and the user's actual level grows.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Case Study Scenario: Targeting Hidden Education Level 🎯

Implicit Goal of the CB Agent

The ad server is implicitly trying to predict the user's **missing education level** as accurately as possible by observing which ad yields the highest click-through rate in a given context ($\mathbf{x}$).

The agent uses the known context (x) to choose the ad (a) that is most likely to "guess" the missing education level and earn a click.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: Data Preparation

See '**Contextual bandits.ipynb**' file in the Teams course page.

- Download the data and select the columns of interest: `age, workclass, education, marital_status, occupation, relationship, race, gender, hours_per_week, native_country, income`
- Drop the rows with missing data, marked by '?' entries:

```
df_census = df_census.replace('?',np.nan).dropna()
```

Note that instead of dropping the missing data we can impute them but this is out of our scope here.

Implementation: Data Preparation

Group different education levels to five categories: `Elementary`, `Middle`, `Undergraduate`, and `Graduate`

```
edu_map = {'Preschool': 'Elementary',
           '1st-4th': 'Elementary',
           '5th-6th': 'Elementary',
           '7th-8th': 'Elementary',
           '9th': 'Middle',
           '10th': 'Middle',
           '11th': 'Middle',
           '12th': 'Middle',
           'Some-college': 'Undergraduate',
           'Bachelors': 'Undergraduate',
           'Assoc-acdm': 'Undergraduate',
           'Assoc-voc': 'Undergraduate',
           'Prof-school': 'Graduate',
           'Masters': 'Graduate',
           'Doctorate': 'Graduate'}
# Assign the result of the replacement operation back to the 'education' column.
df_census['education'] = df_census['education'].replace(edu_map)
```

Implementation: Data Preparation

Next, we **convert** categorical data into **one-hot vectors** to be able to feed into a DNN. We preserve the education column as is, since that is not part of the context:

```
context_cols = [c for c in usecols if c != 'education']
df_data = pd.concat([pd.get_dummies(df_census[context_cols]),
          df_census['education']], axis=1)
```

By doing this conversion at the beginning, we **assume** that **we know** all possible work class categories and native countries.

Implementation: Simulating Ad Clicks

In this example, **the availability of ads is random** and the ad clicks are stochastic. We need to come up with some logic to simulate this behavior.

Let's start with determining the ad availability probabilities for each education category and implement the sampling of ads.

Some ad categories may not be returned. So there is **no guarantee** to have a chance to show them. This is different from the previous scenario where we select among 5 advertiser (A, B, C, D, E) anytime we want!

```
def get_ad_inventory():
    ad_inv_prob = {'Elementary': 0.9,
                   'Middle':  0.7,
                   'HS-grad':  0.7,
                   'Undergraduate':  0.9,
                   'Graduate':  0.8}
    ad_inventory = []
    for level, prob in ad_inv_prob.items():
        if U() < prob:
            ad_inventory.append(level)
    # Make sure there are at least one ad
    if not ad_inventory:
        ad_inventory = get_ad_inventory()
    return ad_inventory
```

Implementation: Simulating Ad Clicks

Then, we define a function to generate a click probabilistically, where the likelihood of a click increases to the degree that the user's education level and the ad's target match.

```python
def get_ad_click_probs():
    base_prob = 0.8
    delta = 0.3
    ed_levels = {'Elementary': 1,
                 'Middle':  2,
                 'HS-grad':  3,
                 'Undergraduate':  4,
                 'Graduate':  5}
    ad_click_probs = {l1: {l2: max(0, base_prob - delta * abs(ed_levels[l1]- ed_levels[l2])) for l2 in ed_levels}
                      for l1 in ed_levels}
    return ad_click_probs
```

```
{'Elementary': {'Elementary': 0.8,
  'Middle': 0.5,
  'HS-grad': 0.20000000000000007,
  'Undergraduate': 0,
  'Graduate': 0},
 'Middle': {'Elementary': 0.5,
  'Middle': 0.8,
  'HS-grad': 0.5,
  'Undergraduate': 0.20000000000000007,
  'Graduate': 0},
 'HS-grad': {'Elementary': 0.20000000000000007,
  'Middle': 0.5,
  'HS-grad': 0.8,
  'Undergraduate': 0.5,
  'Graduate': 0.20000000000000007},
 'Undergraduate': {'Elementary': 0,
  'Middle': 0.20000000000000007,
  'HS-grad': 0.5,
  'Undergraduate': 0.8,
  'Graduate': 0.5},
 'Graduate': {'Elementary': 0,
  'Middle': 0,
  'HS-grad': 0.20000000000000007,
  'Undergraduate': 0.5,
  'Graduate': 0.8}}
```

Implementation: Function Approximation with Neural Networks

- As mentioned, we use a (not so) **DNN**, which will **estimate the action value**, **given the context and the action**.
- The DNN we'll use has two layers, with 256 hidden units in each layer.
- This model is pretty easy to create using Keras, TensorFlow's high-level API.
- We use **dropout** that we **leave active for the inference time** as a Bayesian approximation that we need for Thompson sampling. This is configured by setting **training=True** in the dropout layer.

Implementation: Function Approximation with Neural Networks

- We create a function that returns a compiled DNN model with given input dimensions and a dropout rate (Using Keras Functional API):

```
def get_model(n_input, dropout):
    inputs = keras.Input(shape=(n_input,))
    x = Dense(256, activation='relu')(inputs)
    if dropout > 0:
        x = Dropout(dropout)(x, training=True)
    x = Dense(256, activation='relu')(x)
    if dropout > 0:
        x = Dropout(dropout)(x, training=True)
    phat = Dense(1, activation='sigmoid')(x)
    model = keras.Model(inputs, phat)
    model.compile(loss=keras.losses.BinaryCrossentropy(),
                  optimizer=keras.optimizers.Adam(),
                  metrics=[keras.metrics.binary_accuracy])
    return model
```

The Role of `training=True` (Crucial for Thompson Sampling)
In this specific Contextual Bandit application, the parameter `training=True` is critically important:

•**Normal Behavior:** By default, Keras automatically switches `Dropout` **off** during inference (when calling `model.predict()` or `model()`).

•**Override for TS:** Setting `training=True` **explicitly forces the dropout mechanism to remain active** even when the model is used for prediction. This introduces the random variation needed to perform **Approximate Bayesian Inference** for Thompson Sampling. The output of the prediction becomes stochastic (random), simulating sampling from the model's posterior distribution.

Implementation: Function Approximation with Neural Networks

- We will update this model in batches as data becomes available. Next, write a function to train the model for 10 epochs with each batch:

```
def update_model(model, X, y):
    X = np.array(X)
    X = X.reshape((X.shape[0], X.shape[2]))
    y = np.array(y).reshape(-1)
    model.fit(X, y, epochs=10)
    return model
```

Implementation: Function Approximation with Neural Networks

- We then define a function that returns a one-hot representation for a specified ad based on the education level it targets:

```
def ad_to_one_hot(ad):
    ed_levels = ['Elementary',
                 'Middle',
                 'HS-grad',
                 'Undergraduate',
                 'Graduate']
    ad_input = [0] * len(ed_levels)
    if ad in ed_levels:
        ad_input[ed_levels.index(ad)] = 1
    return ad_input
```

Implementation: Function Approximation with Neural Networks

- We implement the **Thompson sampling** to **select** an **ad** given the **context** and the **ad inventory** at hand.

- The ad to be displayed is chosen based on the **largest action value estimate** that we obtain from the DNN. We obtain this by **trying all available ads** in the inventory – and remember, we have at most one ad per target user group – with the context of the user. **Note that the target user group is equivalent to action**, which we feed to the DNN in the format of a one-hot vector.

Because the `Dropout` layers in the `model` were initialized with `training=True`, each prediction is a **random sample** from the model's uncertainty distribution (the posterior).

```
def select_ad(model, context, ad_inventory):
    selected_ad = None
    selected_x = None
    max_action_val = 0
    for ad in ad_inventory:
        ad_x = ad_to_one_hot(ad)
        x = np.array(context + ad_x).reshape((1, -1))
        action_val_pred = model.predict(x)[0][0]
        if action_val_pred >= max_action_val:
            selected_ad = ad
            selected_x = x
            max_action_val = action_val_pred
    return selected_ad, selected_x
```

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Control Over Randomness: Classic vs. MCDO

Feature	Classic Beta-Bernoulli TS (MAB)	Monte Carlo Dropout TS (CB/NN)
Source of Randomness	Sampling from the **Beta distribution** ($\text{Beta}(\alpha, \beta)$).	Randomly dropping neurons (setting activations to zero) via the **Dropout layer**.
Exploration Control Parameter	The $\boldsymbol{\alpha}$ **and** $\boldsymbol{\beta}$ **counts** (successes and failures). These parameters directly determine the shape and width of the distribution.	The **Dropout Rate** (e.g., 0.1, 0.5).
Link to Uncertainty	Direct. Larger counts mean a narrower Beta distribution (less uncertainty).	Indirect. Higher uncertainty in weights (due to less data) leads to larger variance in the output when dropout is applied.

The Model's Confidence (The Learned Uncertainty)
In Bayesian terms, the degree of output variance is also naturally driven by the model's learned uncertainty in its own weights:
High Uncertainty (New/Scarce Data): When the model's weights have not been strongly fixed by data (e.g., a specific context-ad pair has been seen rarely), applying dropout to those shaky weights produces higher variance in the output Q-value, leading to increased exploration.
Low Uncertainty (Plenty of Data): When the model has seen a context-ad pair many times, the weight estimates are strong. Applying dropout produces lower variance, leading the agent to exploit the reliably high (or low) Q-value.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: Function Approximation with Neural Networks

- We write a function to generate users through a random selection from the dataset. The function will return the user data as well as the derived context :

```
def generate_user(df_data):
    user = df_data.sample(1)
    context = user.iloc[:, :-1].values.tolist()[0]
    return user.to_dict(orient='records')[0], context
```

Implementation: Calculating Regret

- We'll continue to use regret for comparing various versions of the CB algorithm. We calculate it as follows:

```
def calc_regret(user, ad_inventory, ad_click_probs, ad_selected):
    this_p = 0
    max_p = 0
    for ad in ad_inventory:
        p = ad_click_probs[ad][user['education']]
        if ad == ad_selected:
            this_p = p
        if p > max_p:
            max_p = p
    regret = max_p - this_p
    return regret
```

Implementation: Putting All Together

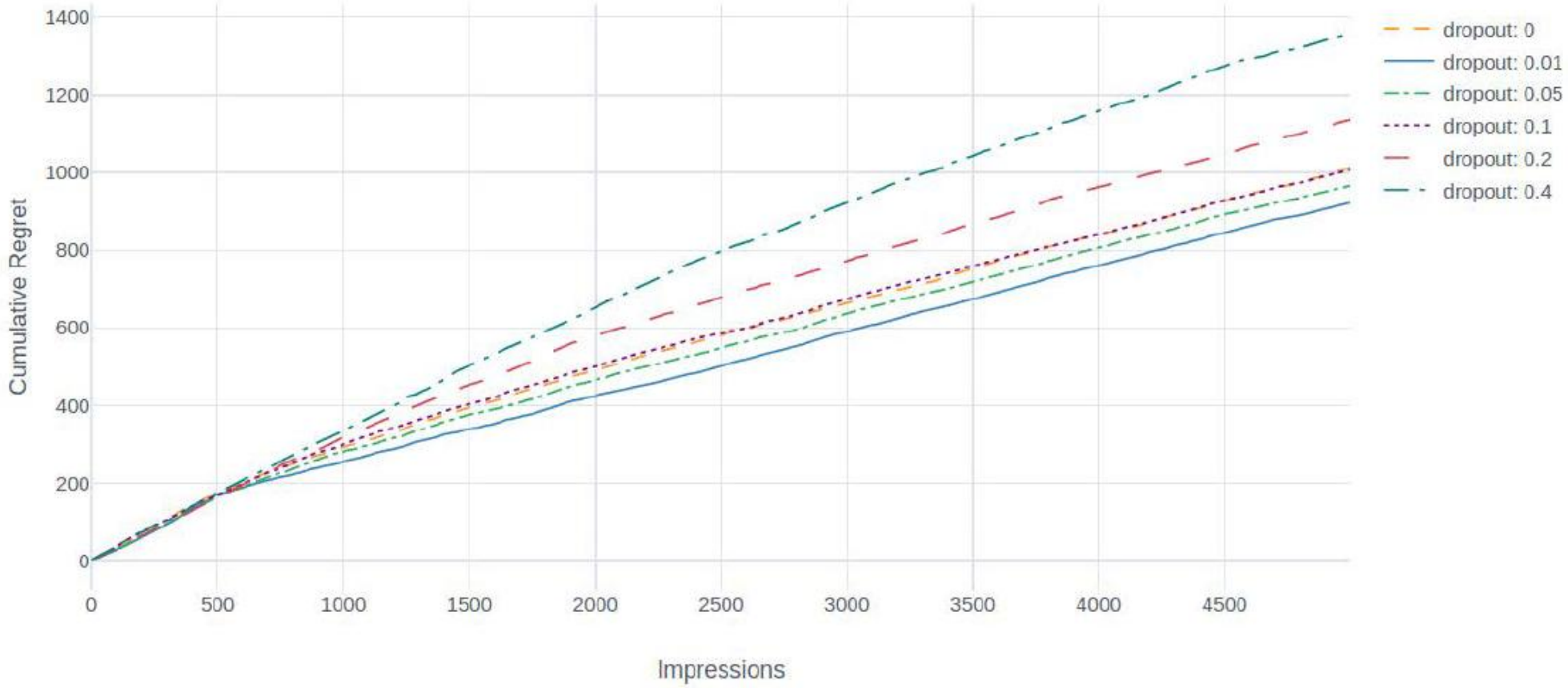
- We'll try this algorithm with different dropout probabilities over 5,000 impressions. We'll update the DNN parameters after every 500 iterations. Here is the implementation in Python for various dropout rates

```
ad_click_probs = get_ad_click_probs()
df_cbandits = pd.DataFrame()
dropout_levels = [0, 0.01, 0.05, 0.1, 0.2, 0.4]
for d in dropout_levels:
    print("Trying with dropout:", d)
    np.random.seed(0)
    context_n = df_data.shape[1] - 1
    ad_input_n = df_data.education.nunique()
    model = get_model(context_n + ad_input_n, 0.01)
    X = []
    y = []
    regret_vec = []
    total_regret = 0
    for i in range(5000):
        if i % 20 == 0:
            print("# of impressions:", i)
        user, context = generate_user(df_data)
        ad_inventory = get_ad_inventory()
        ad, x = select_ad(model, context, ad_inventory)
        click = display_ad(ad_click_probs, user, ad)
        regret = calc_regret(user, ad_inventory, ad_click_probs, ad)
        total_regret += regret
        regret_vec.append(total_regret)
        X.append(x)
        y.append(click)
        if (i + 1) % 500 == 0:
            print('Updating the model at', i+1)
            model = update_model(model, X, y)
            X = []
            y = []

    df_cbandits['dropout: '+str(d)] = regret_vec
```

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: Cumulative Regrets



Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

How to Select the Dropout Rate

Solution 1: Manual Tuning (Empirical Approach)

Method: Try a range of different fixed rates over time (e.g., 0.01, 0.05, 0.1, 0.2) to identify which works best in similar problems.

Pros: Practical and often works well if the business tackles similar problems repeatedly.

Cons: Time-consuming, requires human intervention, and may not find the truly optimal rate for every unique context.

Solution 2: Automated Learning (Concrete Dropout) 🧠

Method: Instead of fixing the rate, **learn the optimal dropout probability** as part of the model training process.

Concrete Dropout is a variant that tunes the dropout probabilities automatically.

Benefit: Studies (e.g., Collier & Llorens, 2018) have successfully used this method in Contextual Bandit problems, reporting **superior performance** over manual, fixed dropout selections.

Tip: For a TensorFlow implementation of Concrete Dropout, refer to the resources available online.

Other applications of multi-armed bandits and CBs

Application: Recommender Systems

Bandits as Recommenders

Bandit algorithms inherently recommend actions (e.g., which ad to display).

Common Use Cases:

Artwork Selection: Choosing thumbnails/images for movie titles (e.g., Netflix).

Content Recommendation: Selecting articles on news portals or posts on social media.

E-commerce: Recommending products or services on retail platforms.

Search: Tailoring search engine results to individual users.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Other applications of multi-armed bandits and CBs

Application: Web Page/App Feature Design

Efficient A/B Testing Alternative

Traditional A/B/n tests often waste resources exploring poorly performing options.

Bandit Approach:

Continuously test different designs for features (e.g., "buy" buttons, layouts, UI elements).

Quickly identify and **eliminate underperforming designs** based on real-time feedback (e.g., conversion rates, sales).

Minimizes user disruption by reducing exposure to bad designs.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Other applications of multi-armed bandits and CBs

Application: Healthcare

Personalized Medicine

Increasing availability of patient data enables personalized treatments.

Contextual Bandits (CBs) help decide which treatment to apply based on patient features (context) in trials.

Drug Dosage Optimization

Successfully used for deciding treatment doses for drugs like Warfarin (blood coagulation).

Efficient Trials

Optimize data sampling allocation across different models (e.g., animal models) to assess treatment effectiveness more quickly.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Other applications of multi-armed bandits and CBs

Applications: Dynamic Pricing, Finance & Control

Dynamic Pricing

Challenge: Adjust prices for millions of products dynamically (online retail).

CB Approach: Model pricing decisions using context like demand forecasts, inventory levels, costs, location.

Finance

Optimal portfolio construction by mixing passive/active investments.

Goal: Balance risk and expected return.

Control Systems Tuning

Automate the tuning of PID controllers in mechanical systems.

CBs can assess system characteristics (material, temp, wear) and adjust controller gains accordingly, replacing manual expert tuning.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Summary & Conclusion

- Bandit algorithms (MABs & CBs) are powerful tools for **single-step** (or seemingly single-step) **decision problems under uncertainty**.
- They have a vast array of practical applications well beyond online advertising.
- Their core strength lies in efficiently managing the exploration-exploitation dilemma online.
- Understanding bandits provides crucial intuition for solving this dilemma, which exists in almost every Reinforcement Learning setting.
- **Next Step**: Moving from single-step bandits to full multi-step RL using Markov Decision Processes (MDPs).

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."